

# Node Stack – Complete 50 L1 Interview Questions

This PDF guide contains 50 commonly asked questions for Node.js L1 technical interviews. It includes soft skills questions, core Node.js and JavaScript theory, practical coding examples, and real-world scenario questions. Each question has a simple, beginner-friendly answer.

## Section 1: HR & Introduction Questions

### **1. Tell me about yourself**

Give a brief introduction: your education, experience, current role, and skills in 2-3 sentences.

### **2. What are your strengths?**

Mention a few skills or traits that are relevant to the role.

### **3. What are your weaknesses?**

Mention a real weakness but also how you are improving it.

### **4. Why do you want to work with us?**

Talk about interest in technology, Node.js, company culture, and learning opportunities.

### **5. Where do you see yourself in 5 years?**

Focus on growth, learning, and contributing to the company.

### **6. Why did you choose Node.js?**

Explain your interest in backend development, scalability, and JS ecosystem.

### **7. Are you comfortable working in a team?**

Yes, give a small example of past teamwork.

### **8. How do you handle challenges at work?**

Explain calmly with an example of problem-solving approach.

### **9. Describe a project you worked on.**

Briefly explain project, tech stack, your role, and outcome.

### **10. Why should we hire you?**

Highlight skills, enthusiasm, and willingness to learn.

## Section 2: Core Node.js & JavaScript Questions

**11. What is Node.js?**

Node.js is a runtime environment that allows JavaScript to run on the server side.

**12. What is npm?**

npm is Node Package Manager to install and manage packages for Node.js.

**13. What is Express.js?**

Express.js is a framework for Node.js that simplifies building servers and APIs.

**14. What is package.json?**

It stores project details, dependencies, and scripts.

**15. What is middleware?**

Middleware is a function that runs between request and response.

**16. Difference between synchronous and asynchronous code?**

Synchronous waits line by line, asynchronous runs in the background.

**17. What is a callback?**

A function passed into another function, called when the task completes.

**18. What is a Promise?**

Represents a future result of async operations: resolved or rejected.

**19. What is async/await?**

Simpler way to write async code that looks like synchronous code.

**20. What is require()?**

Used to import modules into a Node.js file.

**21. Difference between require and import?**

require is CommonJS, import is ES6 module syntax.

**22. What is event loop?**

It allows Node.js to handle many tasks asynchronously without blocking.

**23. What is REST API?**

A way to design web services using HTTP methods to access resources.

**24. What are HTTP status codes?**

Codes like 200, 404, 500 indicate request response status.

**25. What is CORS?**

Security feature controlling which domains can access your API.

**26. What are environment variables?**

Variables outside code storing configuration data like DB URLs.

**27. How to handle errors in Express?**

Use middleware with 4 params: err, req, res, next.

**28. Difference between var, let, const?**

var is function-scoped, let/const are block-scoped; const cannot be reassigned.

**29. What is callback hell and how to avoid it?**

Nested callbacks causing hard-to-read code; use Promises or async/await.

**30. How to debug Node.js code?**

Use console.log, VSCode debugger, or node inspect command.

## Section 3: Practical / Coding Questions

**31. Create a simple HTTP server in Node.js**

Example:

```
const http = require('http');  
http.createServer((req,res)=>res.end('Hello')).listen(3000);
```

**32. Create a simple Express server**

Example:

```
const express=require('express');  
const app=express();  
app.get('/',(req,res)=>res.send('Hello'));  
app.listen(3000);
```

**33. Create a REST API endpoint for /users**

Example using Express:

```
app.get('/users',(req,res)=>res.json([{id:1,name:'John'}]));
```

**34. Read a file asynchronously in Node.js**

```
const fs=require('fs');  
fs.readFile('file.txt','utf8',(err,data)=>console.log(data));
```

**35. Write data to a file**

```
fs.writeFile('file.txt','Hello',(err)=>console.log('Done'));
```

**36. Use a Promise to handle async task**

`new Promise((resolve, reject) => resolve('Done')).then(msg => console.log(msg));`

**37. Convert a callback function to Promise**

Example: wrap `fs.readFile` in a Promise to avoid callback hell.

**38. Create middleware to log requests in Express**

`app.use((req, res, next) => { console.log(req.url); next(); });`

**39. Handle 404 errors in Express**

`app.use((req, res) => res.status(404).send('Not Found'));`

**40. Create a simple CRUD API using Express**

Use `app.get`, `app.post`, `app.put`, `app.delete` with in-memory array as storage.

## Section 4: Scenario & Real-world Questions

**41. How do you secure an API?**

Use authentication (JWT, OAuth), HTTPS, and input validation.

**42. How to handle multiple requests in Node.js?**

Node.js event loop handles requests asynchronously.

**43. How to connect Node.js with database?**

Use drivers or ORM (like Sequelize, Mongoose) to connect to MySQL/MongoDB.

**44. How do you debug memory leaks?**

Use Node.js profiling tools or Chrome DevTools to inspect memory usage.

**45. How to structure a Node.js project?**

Separate routes, controllers, services, models; keep config in separate file.

**46. How do you test Node.js APIs?**

Use tools like Postman or write unit tests with Jest or Mocha.

**47. How to handle large JSON responses?**

Paginate, limit fields, or stream responses.

**48. How to handle async errors in Express?**

Use `try/catch` inside async functions or middleware error handling.

***49. How to improve performance of Node.js server?***

Use clustering, caching, avoid blocking code, and optimize DB queries.

***50. What is your approach to learning new Node.js features?***

Read official docs, practice small projects, follow tutorials, and experiment with code.